

A. (defun f(m)

(cond

((= m 0) 0)

(> (f (- m 1)) 1) (- m 2))

(t (+ (f (- m 1)) 1))

)

)

folosim o funcție lambda

(defun f(m)

(cond

((= m 0) 0)

(t (lambda (x)

(cond

((> x 1) (- m 2))

(t (+ x 1))

)

)(f (- m 1))

)

,

↳ este x din funcția lambda

)

B. dublare - succesori ([H], [H]): -!

dublare - succesori ([H<sub>1</sub>, H<sub>2</sub> | T], [H - REZ | R]): -

H<sub>1</sub> < H<sub>2</sub>, !, H - REZ is 2 \* H<sub>1</sub>,

dublare - succesori ([H

dublare - succesori ([H<sub>1</sub>, H<sub>2</sub> | T], [H<sub>1</sub> | R]): - dublare - succesori ([H<sub>2</sub> | T], R).

egal ([], [ ]): -!

egal ([H<sub>1</sub> | T<sub>1</sub>], [H<sub>2</sub> | T<sub>2</sub>]): - H<sub>1</sub> ::= H<sub>2</sub>, egal (T<sub>1</sub>, T<sub>2</sub>).

maim (L<sub>1</sub>, L<sub>2</sub>): -

dublare - succesori (L<sub>1</sub>, L<sub>2</sub>), -> dacă a făcut dublare și nu s-au produs  
egal (L<sub>1</sub>, L<sub>2</sub>), !. *modificări, ne opriți*

maim (L<sub>1</sub>, Rez): -

dublare - succesori (L<sub>1</sub>, L<sub>2</sub>), -> face dublare și tot repetăm până nu se  
maim (L<sub>2</sub>, Rez). *mai produc modificări*

C. submultimi ([], [ ]).

submultimi ([ - | T], Rez): - submultimi (T, Rez).

submultimi ([H | T], [H | Rez]): - submultimi (T, Rez).

suma ([], 0): -!

suma ([H | T], S): - suma (T, S<sub>1</sub>), S is S<sub>1</sub> + H

condiție - submultimi - suma (L, Rez): - submultimi (L, Rez),

suma (Rez, RezSuma),

RezSuma mod 2 ::= 0.

submultimi - suma (L, Rez): - findall (Template, Goal, Bag)

findall (Submultime Bună, condiție - submultimi - suma (L, Submultime Bună),  
Rez).

D. (defun înlocuire (l)

(cond

((and (numberp l) (equal (mod l 2) 0))

(+ 1 l)

)  
(atom l) l)

(t (mapcar #'înlocuire l))

)

)

→ mapcar grupează rezultatele cu list

176

A.  $f(100, 1) = -1$ .

$f(K, X) = -K_1$  is  $K+1$ ,  $f(K_1, Y)$ ,  $Y > 1$ ,  $K_2$  is  $K_1 - 1$ ,  $X$  is  $K_2 + Y$ .

$f(K, X) = -K_1$  is  $K+1$ ,  $f(K_1, Y)$ ,  $Y > 0.5$ ,  $X$  is  $Y$ .

$f(K, X) = -K_1$  is  $K+1$ ,  $f(K_1, Y)$ ,  $X$  is  $Y - K_1$ .

→ facem o funcție auxiliară

$f(100, 1) = -1$ .

$f(K, X) = -K_1$  is  $K+1$ ,  $f(K_1, Y)$ ,  $\text{aux}(K_1, X, Y)$ .

$\text{aux}(K_1, X, Y) = -Y > 1$ ,  $K_2$  is  $K_1 - 1$ ,  $X$  is  $K_2 + Y$ .

$\text{aux}(\_, X, Y) = -Y > 0.5$ ,  $X$  is  $Y$ .

$\text{aux}(K_1, X, Y) = -X$  is  $Y - K_1$ .

B.  $\text{inserează}(E, L, [E, L])$ .  $\rightarrow$  inserează E pe fiecare poz din L

$\text{inserează}(E, [H1T], [H1Rez])$  :-  $\text{inserează}(E, T, Rez)$ .

$\text{perm}([ ], [ ])$ .

$\text{perm}([E1T], P)$  :-  $\text{perm}(T, L)$ ,  $\text{inserează}(E, L, P)$  } va fi permutarea

$\text{verificare}([ ])$  :- !.  $\rightarrow$  verificare cond. aia pt. o listă

$\text{verificare}([H_1, H_2 | T])$  :-  $H_1 - H_2 = < 3$ ,  $H_1 - H_2 = 0$ , !,

$\text{verificare}([H_2 | T])$ .

$\text{verificare}([H_1, H_2 | T])$  :-  $H_2 - H_1 = < 3$ ,  $H_2 - H_1 = 0$ , !,

$\text{verificare}([H_2 | T])$ .

$\text{condiție}(L, Rez)$  :-  $\text{perm}(L, Rez)$ ,  $\text{verificare}(Rez)$ .

$\text{permutări} - \text{condiție}(L, Rez)$  :-

$\text{findall}(\text{Perm Bună}, \text{condiție}(L, \text{Perm Bună}), Rez)$ .

C. am făcut deja

253

A.  $f(50, 1)$  :- !.

$f(I, Y)$  :-  $Y$  is  $1+1$ ,  $f(Y, S)$ ,  $S < 1$ , !,  $K$  is  $I-2$ ,  $Y$  is  $K$ .

$f(I, Y)$  :-  $Y$  is  $1+1$ ,  $f(Y, Y)$ .

$\rightarrow$  scriem o funcție auxiliară

$f(50, 1)$  :- !.

$\text{aux}(I, Y, S)$  :-  $S < 1$ , !,

$f(I, Y)$  :-  $Y$  is  $1+1$ ,

$K$  is  $I-2$ ,

$f(Y, S)$ ,

$Y$  is  $K$ .

$\text{aux}(I, Y, S)$ .

$\text{aux}(\_, Y, S)$  :-  $Y$  is  $S$ .



```

6. (defun summa-cif (nr)
  (cond
    ((= nr 0) 0)
    (t (+ (mod nr 10) (summa-cif (/ (- nr (mod nr 10)) 10)))))
  )
)

```

```

(defun main-aux (l, minr)
  (cond
    ((null l) nil)
    ((AND (equal (mod minr 2) 1) (numberp (car l)))
      (cons (summa-cif (car l)) (main-aux (cdr l) minr)))
    (atom (car l))
      (cons (car l) (main-aux (cdr l) minr)))
    (t (cons (main-aux (car l) (+ minr 1)) (main-aux (cdr l) minr)))
  )
)

```

```

(defun main (l)
  (main-aux l 1)
)

```

C. submultimi ([], []).

submultimi ([-IT], Rez): - submultimi (T, Rez).

submultimi ([HIT], [HIRE]): - submultimi (T, Rez).

suma ([], 0): -!

suma ([HIT], Rez): - suma (T, Rez1), Rez is Rez1 + H.

nr\_nr\_impere ([], 0): -!

nr\_nr\_impere ([HIT], Rez): -

1 is H mod 2, nr\_nr\_impere (T, Rez1), Rez is Rez1 + 1.

nr\_nr\_impere ([-IT], Rez): - nr\_nr\_impere (T, Rez).

conditie (L, Rez): - submultimi (L, Rez),

nr\_nr\_impere (Rez, R1), 1 is R1 mod 2,

suma (Rez, R2), 1 is R2 mod 2.

submultimi\_maim (L, Rez): -

findall (submBuna, conditie (L, submBuna), Rez).

D. (defun k\_0 (l <sup>→ apelez cu 0</sup> nro k)

(cond

((AND (atom l) (equal nro k)) 0)

((atom l) l)

(t (mapcar #'lambda (x)

(k\_0 x (+ 1 nro k))

) l

(defun maim (l k)

(k\_0 l 0 k)

A. (defun f (l1 l2)  
 (append (f (car l1) l2)  
 (cond  
 ((null l1) (cdr l2))  
 (t (list (f (car l1) l2) (car l2))))  
 )  
 )

→ scriem o funcție lambda

(defun f (l1 l2)  
 (lambda (x)  
 (append x  
 (cond  
 ((null l1) (cdr l2))  
 (t (list x (car l2))))  
 )  
 )  
 ) (f (car l1) l2)  
 )

B. a 25 câ nu dă arbori prileg Doamne ajută!



C.  $\text{insereaza}(E, L, [E|L])$ .

$\text{insereaza}(E, [H|T], [H|Rez]) :- \text{insereaza}(E, T, Rez)$ .

$\text{aranj}([H|_], 1, [H])$ .

$\text{aranj}([_ | T], K, R) :- \text{aranj}(T, K, R)$ .

$\text{aranj}([H|T], K, R) :- K > 1, K1 \text{ is } K-1,$

$\text{aranj}(T, K1, R1),$

$\text{insereaza}(H, R1, R)$ .

$\text{produs}([ ], 1) :- !$ .

$\text{produs}([H|T], R) :- \text{produs}(T, R1), R \text{ is } R1 * H$ .

$\text{conditie}(L, k, P, Rez) :- \text{aranj}(L, k, Rez),$

$\text{produs}(Rez, RezProdus), RezProdus \text{ is } P$ .

$\text{aranjProdus}(L, k, P, Rez) :-$

$\text{findall}(\text{AranjBun}, \text{conditie}(L, k, P, \text{AranjBun}), Rez)$ .

D.  $(\text{dajun apartime-arbore}(l \text{ elem}))$

(cond

$((\text{null } l) \text{ nil})$

$((\text{listp } (\text{car } l)) \text{ (OR } (\text{apartime-arbore}(\text{car } l) \text{ elem}) (\text{apartime-arbore}(\text{cdr } l) \text{ elem})))$

$((\text{equal } (\text{car } l) \text{ elem}) t)$

$(t (\text{apartime-arbore}(\text{cdr } l) \text{ elem}))$

$(\text{dajun path}(l \text{ elem}))$

(cond

$((\text{atom } l) \text{ nil})$

$((\text{and } (\text{listp } l) (\text{apartime-arbore } l \text{ elem}))$

$(\text{append } (\text{list } (\text{car } l))$

$(\text{mapcar } \#'\text{lambda}(x) (\text{path } x \text{ elem}) l)$



A.  $f([], 0).$   
 $f([H|T], S) :- f(T, S1), S1 < H, !, S is H.$   
 $f([_|T], S) :- f(T, S1), S is S1.$

→ predicat auxiliar

$f([], 0).$

$aux(H, S, S1) :- S1 < H, !, S is H.$

$f([H|T], S) :- f(T, S1), aux(H, S, S1).$

$aux(_, S, S1) :- S is S1.$

B. (defun limiarizare (l)

(cond

((null l) nil)

((atom (car l)) (append (list (car l)) (limiarizare (cdr l))))

(t (append (limiarizare (car l)) (limiarizare (cdr l))))

)

(defun get-lista (l) (cond

((or (null (car l)) (null (cdr l)) (null (caddr l))) nil)

((and (car-atom, nu nr cadr-nr caddr-atom, nu nr)  
 (append (list (cadr l)) (get-lista (caddr l)))))

(t (get-lista (cdr l))))

(defun cmmdc (a b)

(cond

((= 0 b) a)

((= a b) a)

((> a b) (cmmdc b (- a b)))

(t (cmmdc a (- b a)))

(defun cmmdc-lista (l)

(cond

((null (cdr l)) (car l))

(t (cmmdc (car l) (cmmdc-lista (cdr l)))))

(defun combina (l)

(cmmdc-lista (get-lista (limiarizare l))))

C. submultimi ([I], [I]).

submultimi ([I-1], Rez) :- submultimi (I, Rez).

submultimi ([HIT], [HIT Rez]) :- submultimi (I, Rez).

suma ([I], 0) :- !.

suma ([HIT], S) :- suma (I, S1), S is S1+H.

conditie (L, Rez) :- submultimi (L, Rez),

suma (Rez, Rez Suma),

0 is Rez Suma mod 2.

main (L, Rez) :- findall (SubmBuna, conditie (L, SubmBuna), Rez).

D. (defun elimina-3 (l) (cond

((or (and (numberp l) (not (equal (mod l 3) 0)))

(and (atom l) (not (numberp l))))

)  
(list l)  $\rightarrow$  ca să nu pună nil când găsește un nr. de  
eliminat  
(listp l) (list (apply 'append (mapcar #'elimina-3 l)))  
(t nil)

(defun main (l)  $\rightarrow$  pt. că o să înfi pună tot într-o listă (în plus)  
la final  
(car (elimina-3 l))

)

A. (defun f(l)

```
(cond
  ((null l) 0)
  ((> (GL) 2) (+ (GL) (f (cdr l))))
  (t (GL)))
)
```

→ scriem o funcție lambda

(defun f(l)

```
(cond
  ((null l) 0)
  (t ((lambda (x)
        (cond
          ((> x 2) (+ x (f (cdr l))))
          (t x)
        ))
      (GL)))
)
```

,

>

B. nu se mai da

C. submultimi  $([I], [I])$ .

submultimi  $([LIT], Rez) :-$  submultimi  $(T, Rez)$ .

submultimi  $([HIT], Rez) :-$  submultimi  $(T, Rez)$ .

elimină-el  $([I], -, [I]) :- !$ .

elimină-el  $([ELIT], EL, R) :-$  elimină-el  $(T, EL, R)$ .

elimină-el  $([HIT], EL, [HIR]) :-$  elimină-el  $(T, EL, R)$ .

minim - listă  $([E], E) :- !$ .

minim - listă  $([HIT], R) :-$  minim - listă  $(T, R1), R$  is mîm  $(H, R1)$ .

sortează

sortează  $(L, [Mim | R]) :-$  minim - listă  $(L, Mim)$ , elimină-el  $(L, Mim, R1)$ ,  
sortează  $(R1, R)$ .

are\_minim - 2  $([-, -1-]) :- !$ .

condiție  $(L, R) :-$  submultimi  $(L, Rez)$ ,  
sortează  $(Rez, R)$ ,  
are\_minim - 2  $(R)$ .

mîm  $(L, Rez) :-$

findall (Subm Bună, condiție  $(L, Subm Bună)$ , Rez).



D. (defun imăltime-mod (l elem găsit)

(atom l) -1)

(AND (listp l) (equal găsit nil) (not (equal (car l) elem)))

dacă nu l-am găsit  
și nici acum nu-l găsim  
doar continuăm

(apply #'max (mapcar

#'(lambda (x)

(imăltime-mod x elem nil)

) l

)

)

)

(t (+1 (apply #'max (mapcar

#'(lambda (x)

(imăltime-mod x elem nil)

) l

)

)

)

)

(defun main (l elem)

imăltime-mod l elem nil)

)

A.  $f(50, 1) :- !$ .

$f(I, Y) :- Y \text{ is } 1+1, f(Y, S), \text{aux}(I, Y, S).$

$\text{aux}(I, Y, S) :- S < 1, !$

$K \text{ is } 1-2, Y \text{ is } K.$

$\text{aux}(-, Y, S) :- Y \text{ is } S.$

B. nu mai dăm

C.  $\text{comb}([H|_], 1, [H]).$

$\text{comb}([_]|T], K, C) :- \text{cond}(T, K, C).$

$\text{comb}([H|T], K, [H|C]) :- K > 1, K1 \text{ is } K-1, \text{comb}(T, K1, C).$

$\text{suma}([], 0) :- !$

$\text{suma}([H|T], S) :- \text{suma}(T, S1), S \text{ is } S1+H.$

$\text{conditie}(L, K, Rez) :- \text{comb}(L, K, Rez),$

$\text{suma}(Rez, RezSuma),$

$0 \text{ is } RezSuma \bmod 2.$

$\text{main}(L, K, Rez) :-$

$\text{findall}(\text{CombBunã}, \text{conditie}(L, K, \text{CombBunã}), Rez).$

D. (defun nr-mir-k (l k mir)

(cond

((AND (atom l) (equal k mir)) 1)

((atom l) 0)

(t (apply +

(mapcar #'(lambda (x)

(nr-mir-k x k (+1 mir)

) l

)

)

)

)

)

(defun main (l k)

(nr-mir-k l k -1)

)

259

D. (defun elimina-numum-pare (l mir)

(cond

((AND (equal (mod mir 2) 0) (atom l) (not (numberp l)))  
nil)

((atom l) (list l))

(t (list (apply #'append (mapcar #'(lambda (x)

(elimina-numum-pare x (+1 mir)

) l

)

(defun main (l)

(car (elimina-numum-pare l 0))

)

